



A Survey of Ethereum Smart Contract Security: Attacks and Detection

TENGYUN JIAO, Swinburne University of Technology, Melbourne, Australia

ZHIYU XU, Swinburne University of Technology, Melbourne, Australia

MINFENG QI, Swinburne University of Technology, Melbourne, Australia

SHENG WEN, Swinburne University of Technology, Melbourne, Australia

YANG XIANG, Swinburne University of Technology, Melbourne, Australia

GARY NAN, Hyper Laboratory, Melbourne, Australia

A smart contract is a computerised transaction agreement that carries out predefined terms without human involvement or third-party intermediaries. It serves as a trust intermediary in several industries, including finance, insurance, and supply chain management, in the blockchain 2.0 era. With the increasing interest in smart contracts, security has become a serious problem. Examining typical vulnerability types and vulnerability detection methodologies is of special importance. In this research, a comprehensive evaluation of common smart contract security vulnerabilities is conducted, and a three-tier threat model is then provided to classify the vulnerabilities. In addition, we examine fourteen existing smart contract analysis tools for finding vulnerabilities and classify them according to the main technique they apply. This article is designed to serve as a reference for people who wish to analyse deployed code and enhance existing detection techniques. At the conclusion, open issues and future research paths regarding smart contract vulnerability detection are presented.

CCS Concepts: • **Security and privacy** → **Distributed systems security**; **Vulnerability scanners**; • **General and reference** → **Surveys and overviews**;

Additional Key Words and Phrases: Blockchain, Ethereum smart contract, security, vulnerability detection

ACM Reference Format:

Tengyun Jiao, Zhiyu Xu, Minfeng Qi, Sheng Wen, Yang Xiang, and Gary Nan. 2024. A Survey of Ethereum Smart Contract Security: Attacks and Detection. *Distrib. Ledger Technol.* 3, 3, Article 23 (September 2024), 28 pages. <https://doi.org/10.1145/3643895>

1 INTRODUCTION

In recent years, blockchain technology has been among the most cutting-edge technologies. The concept of Blockchain was originated with Bitcoin proposed by Satoshi Nakamoto in 2008 [65]. Bitcoin, the first implementation of Blockchain technology, is a decentralised, peer-to-peer-based cryptocurrency. In the first Bitcoin transaction in 2010, 10,000 Bitcoin were exchanged for two slices of pizza. In January 2022, Bitcoin's price unexpectedly reached a record high of around \$68,000 [23]. The Blockchain technology, which serves as the foundation for Bitcoin, has garnered significant attention from both academic and industrial circles, particularly in light of its increasing value. Blockchain is a type of distributed ledger database that possesses unique features,

Authors' addresses: T. Jiao, Z. Xu (Corresponding author), M. Qi, S. Wen, and Y. Xiang, Swinburne University of Technology, John St, Hawthorn, Melbourne, Australia; e-mails: tjiao@swin.edu.au, zhiyuxu@swin.edu.au, minfengqi@swin.edu.au, swen@swin.edu.au, yxiang@swin.edu.au; G. Nan, Hyper Laboratory, Glen Eira Road, Caulfield North, Melbourne, Australia; e-mail: garynan@hyperbc.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2769-6472/2024/09-ART23

<https://doi.org/10.1145/3643895>

including decentralisation, tamper-proof, anonymity, and traceability. During the early stages of blockchain technology's evolution, commonly referred to as the blockchain 1.0 epoch, the primary application was for digital currency. Later, with the birth of Ethereum and smart contract, several **decentralised applications (DApps)** such as MakerDao [56] and Uniswap [2] were developed and implemented, heralding the introduction of Blockchain 2.0.

Nick Szabo, a computer scientist and cryptographer, first proposed the concept of smart contract in 1993 [83], but it didn't gain traction until the advent of Ethereum, which provides a trustworthy **Ethereum Virtual Machine (EVM)** for executing smart contracts. In essence, a smart contract is an automated computer-based trade agreement that operates without the need for a trusted third party's involvement or oversight. Once specific pre-established criteria have been fulfilled, the corresponding contractual provisions shall be implemented in an irreversible manner. Finance [88], supply chain management [91], insurance [9], the **Internet of Things (IoTs)** [37], healthcare [39], and other areas have implemented smart contracts.

Evidently, smart contracts are essentially software programs that possess inherent weaknesses, rendering them highly vulnerable to potential security breaches. The immutability of smart contracts can lead to significant gains for attackers who exploit vulnerabilities in deployed smart contracts. In 2016, attackers exploited the re-entrancy issue to steal over 3.6 million Ethers from the **Decentralized Autonomous Organization (DAO)** [59]. Due to the enormous impact of this attack, the Ethereum community was eventually compelled to perform a hard fork to compensate investors for their losses, giving rise to Ethereum Classic.

Ethereum [16] is the earliest and most developed blockchain platform that supports Turing-complete in the world. In addition to Ethereum, there are various other blockchain platforms that can also support smart contracts, such as Hyperledger Fabric [5], EOS [97], and Stellar [48]. These platforms have their own specific languages. For example, Ethereum provides a contract-oriented, JavaScript-like language called Solidity [22], while Fabric supports smart contracts written in general programming languages such as Java, Go, and Node.js, rather than being limited to domain-specific languages. This study aims at investigating the security of smart contracts on the Ethereum network. The present factors contributing to security vulnerabilities in Ethereum smart contracts are as follows:

- The credibility of a smart contract comes from its tamper-proof nature, which means that the contract cannot be modified once it has been deployed. However, if there are vulnerabilities in the contract, contract developers cannot avoid the degradation of security problems by bug fixes, as is the case with traditional program. This will severely impair the economic value of the contract and public confidence.
- In the development of smart contract projects, disclosing the source code has become a standard practice. The majority of smart contract projects will prefer to disclose their project codes to increase user trust. Although open-source code can raise transparency to avoid bad developer behaviour, it dramatically reduces the cost of an attack for hackers, who can easily acquire the source code.
- There may be problems during the development phase of smart contracts. Currently, the use of smart contracts is still in its infancy. In addition to contract language deficiencies, developers may lack professional contract coding and use knowledge. Additionally, scientific testing techniques are minimal.

In light of the security concerns surrounding smart contracts, various security analysis tools have been developed in recent years to detect potential vulnerabilities in smart contracts and mitigate the risk of attacks. An analytical examination of attack modes via a comprehensive overview of vulnerabilities constitutes a noteworthy contribution of this study. Several similar works are introduced below. Nicola et al. [8] analysed twelve Ethereum smart contract vulnerabilities and provided code samples to illustrate the idea and exploitation of vulnerabilities. It is an early study of related fields; hence, the scope of vulnerabilities is insufficiently covered, and associated protection strategies are not introduced. Chen et al. [18] analysed the Ethereum system's vulnerabilities, threats, and defences in a systematic manner and provided their own insights. It splits 40 vulnerabilities, 29 attacks, and 51 defence techniques into subcategories. Xiao et al. [51] performed the first systematic examination of the

security vulnerabilities of popular blockchain systems and examined a number of actual attack cases. In addition to focusing on the smart contract's vulnerabilities, the article examined additional destructive behaviours, such as criminal activity. In addition, several similar studies [47, 84, 86] provided a comprehensive understanding of the vulnerabilities of smart contract. Compared to the aforementioned articles, this survey provides a full evaluation of current state-of-the-art smart contract security analysis tools based on their applied methodology and vulnerability assessments.

In summary, the following contributions have been made in this work:

- A thorough review of common security vulnerabilities is conducted, and a three-tier threat model is proposed accordingly.
- A comprehensive comparison of smart contract security analysis tools is performed.
- Open challenges and research directions regarding smart contract vulnerability detection are discussed.

The survey is organised as follows. In Section 2, the fundamental ideas of blockchain technology, Ethereum, and smart contract are introduced. Section 3 describes common Ethereum smart contract vulnerabilities. Section 4 categorises the principal techniques and related tools for identifying smart contract vulnerabilities. Section 5 discusses the current research challenges and future research directions. The last section concludes our work.

2 BACKGROUND

2.1 Blockchain System

The year 2008 marked the introduction of a decentralised digital currency system and its corresponding technological specifications, as presented by Satoshi Nakamoto in his article. [65]. The depicted blockchain is a decentralised distributed ledger system that integrates a number of mathematical and cryptographic concepts. It enables the management of digital assets and property rights certificates, as well as their transfer and payment, without the need for a trusted third party. Compared to the conventional centralised ledger system, this approach is completely transparent and tamper-proof.

The consensus algorithm is incorporated into the blockchain system to ensure anonymity and privacy. Each node in a distributed network organises and verifies transactions received from other nodes in the network, then packages them into a block. The consensus mechanism is responsible for enabling rapid consensus among all nodes in a decentralized network, facilitating transaction validation and confirmation.

Currently, the most prevalent consensus algorithms are as follows: **proof of work (PoW)** [65], **proof of stake (PoS)** [49], and **proof of authority (PoA)** [66]. Here we briefly introduce each algorithm's essential logic:

2.1.1 PoW. The PoW consensus algorithm is a widely adopted approach employed in the domain of blockchain technology. The primary objective of its design is to mitigate the problem of double-spending in decentralised networks, thereby ensuring the security and consistency of the blockchain.

The PoW algorithm involves a competition among nodes within the network, referred to as miners, to validate transactions and generate new blocks by solving a complex mathematical problem. The present mathematical puzzle is predicated on a cryptographic hash function that necessitates miners to discover a unique hash value through iterative attempts at various inputs, such that the hash value conforms to specific criteria. The process mentioned above is commonly referred to as mining.

The fundamental concept underlying the PoW algorithm is to ensure the integrity and safety of the blockchain network by utilising computational resources. The blockchain's inherent security mechanism is based on the fact that each block in the chain contains the hash of the previous block. Any attempt to manipulate a block in the chain would require the recalculation of all subsequent blocks, which is a computationally demanding task that would result in substantial expenses for the attacker.

2.1.2 PoS. In contrast to the PoW algorithm, the PoS algorithm does not depend on computational power. Instead, it elects the validators based on the node's equity and ascertains the individual who possesses the authority to validate and create a new block.

The PoS algorithm necessitates that nodes lock up a specific quantity of cryptocurrency as collateral or equity to prove their commitment and involvement in the network. Subsequently, the selection of a validator to generate and authenticate the latest block is dependent on the amount of collateral held by a particular node. The selection of a validator in a PoS algorithm may vary depending on the specific implementation and can be based on factors such as random selection or the quantity of collateral involved.

PoS algorithms offer various benefits in comparison to PoW algorithms. Initially, PoS algorithms require minimal computational resources and energy consumption, thereby potentially minimising the energy consumption of the network. In addition, PoS algorithms serve to mitigate the potentiality of a 51% attack being executed by a malicious actor, given that said actor would be required to possess a significant proportion of the cryptocurrency in question in order to attain control. Furthermore, the PoS algorithm enhances the scalability of the network by enabling faster block creation without the need for mining competition.

2.1.3 PoA. The PoA algorithm prioritises the verifier's identity and authority over their computational power. In contrast to the PoW and PoS algorithms, the validators in the PoA algorithm are designated entities or nodes that possess a verified identity and established trust.

The PoA algorithm involves a verification process whereby the legitimacy of the verifier is established through the provision of identity information, which may include a digital signature, a public key, or other means of authentication. Upon receiving authorisation, the verifier is empowered to generate and authenticate novel blocks, subsequently appending them to the blockchain. The act of granting authorisation can be executed by a centralised entity, which may take the form of an organisation, institution, or designated node possessing both trust and control over the verifier.

The PoA algorithm offers a significant benefit in terms of its ability to achieve high throughput and speed up transaction confirmation. Due to their authorisation and trustworthiness, validators are capable of efficiently achieving consensus and validating transactions, thereby expediting the blockchain's processing. Furthermore, PoA algorithms have the potential to offer enhanced security measures due to the authentication and supervision of verifiers, who are tasked with ensuring the network operates correctly.

PoA algorithms are commonly employed in private or federated blockchain networks that are typically governed and supervised by a designated organisation or entity. Several blockchain initiatives that utilise PoA algorithms comprise of Ether's Clique algorithm, Quorum, and various others. These endeavours utilise the PoA algorithm to offer a proficient and safeguarded consensus mechanism in particular application scenarios.

2.2 Ethereum

Bitcoin was the first decentralised peer-to-peer electronic cash system, but its design and scripting language lacked Turing-completeness to handle complex computations. To enhance Bitcoin, Vitalik Buterin created Ethereum [16], a groundbreaking blockchain platform that allows smart contracts and decentralised applications. Ethereum is a public blockchain that implements EVM, a Turing-complete virtual machine that runs on every entire network node. The concept of Turing-completeness refers to the ability of a system to perform any type of information processing that a computer can execute. The EVM facilitates decentralised application development by enabling processing and confirmation of operations on every node in the network.

In Ethereum network, there are two types of accounts: external accounts and contract accounts [94]. Both accounts are 20 bytes long, which includes the unique address, current balance, data storage, nonce (a random number), and contract address (if any). The private key governs the **external owned account (EOA)** and the address is generated after the public key has been encrypted. It can send transactions with encrypted signatures. The contract account is created when the contract is created, and the address is generated simultaneously. Besides,

only contract account has code, which can be invoked by executing a specific code sequence in accordance with predefined rules in order to perform certain actions.

Within the EVM, Gas represents the expenditure associated with executing a transaction on the Ethereum blockchain, which is mandatory for the sender of said transaction to cover [4]. The simpler the transaction, such as a simple Ethereum transfer transaction, the fewer steps are required, and the gas requirement will be lower. Conversely, when performing complex tasks, there will be a significant increase in gas consumption. Subsequent to this, we shall expound upon the gas mechanism in the ensuing segment. In addition to offering EVM for the execution of contract code, Ethereum provides a contract-oriented programming language (Solidity) for contract development. Solidity is currently one of the most popular smart contract languages (besides, there are also Vyper [44], Rust [58], Bamboo [1], Lolisa [99], etc.). Using the Solidity programming language, developers can create their own contracts based on their business logic. Following the completion of development, the contract can be compiled to bytecode and executed on the EVM. Then the contract will be deployed to the Ethereum network using a single deployment transaction, where it will be stored permanently.

2.3 Smart Contract

A smart contract is a digital agreement that fulfils specified terms without human involvement or third-party intermediaries. In contrast to conventional program, the life cycle of a smart contract typically consists of the following five stages:

- Develop: First, developers convert contract logic into code using high-level languages such as Solidity. Solidity is a procedural (imperative) programming language with a syntax similar to Javascript and C++. Originally developed by Gavin Wood, Solidity is the most popular language for Ethereum smart contracts. Therefore, the majority of current Ethereum smart contract security analysis tools are designed for Solidity.
- Compile: After the developer completes the contract coding in a high-level language, the code will be compiled into low-level bytecode supported by the Ethereum virtual machine before it can be executed. The ABI is also generated at the same time. ABI is the interface description of the contract program, which includes information such as function name, field type, parameter name, parameter type, function return value, and so on. ABI is the required interface for interacting with bytecode.
- Deploy: In addition to the bytecode and ABI that will be published on the Ethereum blockchain after the compilation phase, a specific contract initiation transaction is also created. To confirm this transaction, miners need to perform a hash calculation operation (mining) to package it into a block. The bytecode of the contract will be stored in the data field of this transaction. A unique contract account address will be established as the contract's identifier when miners create a contract account. This address is derived using the sender's address and the nonce value, and it is returned to the sender once the transaction has been confirmed.
- Execution: After the contract is deployed, the user can use the contract address obtained in the previous stage to invoke the contract, which is the execution process of the contract. The invocation of the contract can be initiated in two forms: transaction and message call [62]. A message call is a read-only local invocation of a contract function that doesn't change anything on the blockchain and therefore won't cost Ether [94]. Transactions are processed and packaged by the miners and will be published to the blockchain network, which might change the state of the blockchain. It is a write-operation and costs a certain amount of Ethers. Different invoking methods are suitable for different use scenarios, and also have different mechanisms and code logic. Security risks can arise if contract developers fail to adequately consider different situations.
- Self-Destruct: The EVM provides a Self-destruct OPCODE [20]. The self-destruct operation allows developers to send the current balance of a contract to a specified address if there is a problem with the contract or if they want to terminate it. The developer needs to specify when to destroy the contract if certain

conditions are met. After the self-destruct operation, the bytecode and storage of the contract will be removed from all subsequent blocks, the contract will no longer be able to be invoked, the transaction made to the contract will fail, and any funds sent to the destroyed contract will be irretrievably lost. Consequently, the self-destruct feature may potentially be utilised by adversaries. Developers should be aware that to perform self-destruct function, they must have adequate permissions; otherwise, it could result in DoS attacks or other undesirable outcomes.

Smart contracts offer a range of benefits on blockchain networks, such as Ethereum, which include decentralisation, automated execution, tamper-proof features, openness, and programmability.

- Decentralisation: The execution of smart contracts on the blockchain eliminates the requirement of relying on a centralised third-party entity, thereby establishing a decentralised system. Decentralised networks enable the execution of transactions and business logic without the involvement of intermediaries, thereby offering enhanced transparency and dependability.
- Automation: Smart contracts are capable of performing predetermined operations in an automated manner without the need for human involvement. The implementation of this measure mitigates the likelihood of human fallibility and deceit while simultaneously enhancing the efficacy of transactions.
- Immutability: The tamper-proof nature of smart contracts is attributed to their deployment on the blockchain, which ensures that any alteration to the code or state of the contract is easily detectable. This offers an elevated degree of security and reliability, guaranteeing that transactions and commercial regulations remain impervious to tampering.
- Openness and programmability: Smart contracts rely on official protocols, such as Ethereum's ERC20 and ERC721, and possess the capability to engage with additional smart contracts and applications. This enables software developers to construct intricate DApps and protocols.

Although smart contracts provide numerous advantages, it is imperative to acknowledge their limitations, which encompass:

- Limited functionality: The functions of a smart contract are established by the underlying logic and regulations encoded within the contract's code. Upon deployment onto the blockchain, smart contracts are restricted to executing solely the predetermined terms encapsulated within them, avoiding any additional operations. The contractual function is immutable and lacks the ability to be modified in real-time during its execution. It can be inferred that contracts lack the ability to independently react to external stimuli or modify their actions accordingly. In the event that modifications or expansions to functionality are required, the typical course of action involves the deployment of fresh contracts or engagement with existing contracts.
- Lack of standardization: Various blockchain platforms and development frameworks may have their own standards and toolkits for smart contract development at this time. The Ethereum platform, for instance, uses the Solidity programming language and the ERC standard, whereas other blockchain platforms may employ various languages and specifications. This lack of standardisation makes it more difficult to migrate smart contracts or interact with them across platforms.
Smart contracts may not interact seamlessly across blockchain platforms due to the lack of uniform standards. This implies that a smart contract developed on Ethereum may not function directly on other platforms or may require additional adaptations and modifications. The incorporation of cross-chain applications and the integration of multiple blockchains is accompanied by an increase in technical intricacy and financial investment.
- Dependence on underlying blockchain technology: Smart contracts are built on top of the blockchain protocol. The execution and security of a smart contract are dependent on the security and dependability of the protocol that it is built upon. If the fundamental blockchain technology is vulnerable or attacked, smart contract execution and asset security may be jeopardised.

The consensus algorithm employed by the underlying blockchain technology is crucial to the security of smart contracts. Consensus algorithms ensure that all network nodes concur, verify, and log all transactions. If the consensus algorithm is flawed or attacked, the implementation of smart contracts may be subject to errors or fraud.

- Complexity: Developing and auditing smart contracts requires corresponding background knowledge, and the methods and tools for security assessments and audits may also differ due to the previously mentioned lack of standardization in smart contracts. This can lead to differences in contract security and vulnerability detection across platforms, requiring the use of specialized knowledge to perform individualized assessments and audits.

3 SMART CONTRACT VULNERABILITIES

Ethereum smart contracts were endowed with many unique programmatic characteristics from their inception. The programmatic features confer trustworthy attributes on smart contracts but also bring unprecedented security risks. For instance, in previous attacks, attackers exploited Ethereum's gas mechanism to steal wealth. In this section, we first introduce these features and then discuss smart contract vulnerabilities.

- Gas Mechanism: Ethereum, similar to a high-performance computing system, incorporates a functional ecosystem known as the EVM within its blockchain infrastructure. Individuals possessing an Ethereum account are able to utilise a computer system to execute various functions, including but not limited to the implementation and activation of intelligent contracts, the transfer of monetary resources, and other related actions. The process of verifying transactions and consolidating them into blocks is a crucial responsibility for miners. These operations require significant computational resources. As a result, Ethereum includes a gas mechanism to measure the consumption of diverse resources and prevent resource waste from disrupting normal operations. Any Ethereum OPCODE is accompanied by a gas cost proportional to its computational complexity [57]. When starting a contract call, the user must define the maximum amount of gas that can be cost and the price per unit of gas that they are willing to pay. The transaction fee that the user chooses to pay will affect the time spent on transaction packaging. The Gas mechanism prevents the smart contract from entering an infinite loop and abusing resources. The EVM will issue an Out-of-gas exception [33] to stop contract execution if the execution cost of the contract program exceeds the maximum cost and does not stop. In addition, it can encourage more miners to join the Ethereum network, thereby enhancing the system's security and performance. However, malevolent actors may also exploit the gas mechanism to perform denial-of-service attacks against contracts.
- Delegatecall: Contracts can call other contracts through a message call. Ethereum smart contracts provide a special type of message call mechanism called Delegatecall. Delegatecall is an external function call in which the instructions in the external function use and modify the caller's context information [42]. The fundamental distinction between this instruction and the conventional call instruction is that the ordinary call instruction jumps to the callee contract, executes the associated code, and then returns to execute the following code, which has no effect on the context of the calling contract. Delegatecall can facilitate the implementation of the function libraries, compensating for the limitations caused by the immutability of the contract code. However, once the attackers own the target address of the Delegatecall, they may possibly have the ability to take arbitrary actions on the contract, posing a significant security concern.
- Global state and Call sequence: The global namespace of Ethereum smart contracts defines a number of special variables for storing the contracts' global variable states, which can be accessed across functions [93]. Control flow analysis is a crucial method of information gathering in the field of traditional program analysis. However, some smart contract processes might need a series of calls made up of several transactions to be finished. This feature causes multiple function entries in smart contracts, a lack of control flow information between processes, and the possibility that obtaining control flow information only within

Table 1. Smart Contract Vulnerabilities in Three Tiers

Layer	Vulnerabilities	Threats Detail
Blockchain layer	3.1 Timestamp dependency	related to the characteristics of the blockchain itself
	3.2 Transaction-ordering dependency	
EVM layer	3.3 Reentrancy	related to VM and bytecode design specification, VM implementation, and so on.
	3.4 Short address	
Solidity layer	3.5 Mishandled exception	related to high-level language design patterns, user program writing, and so on.
	3.6 Integer overflow/underflow	
	3.7 Dangerous delegatecall	
	3.8 Frozen ether	
	3.9 Tx.origin	
	3.10 Denial of service	
	3.11 Uninitialized Pointer	

functions may not be sufficient [21]. These issues make it difficult to automatically detect vulnerabilities in smart contracts.

Leveraging the above-introduced features is the mainstream method for attackers to exploit smart contract vulnerabilities. In this section, we analyze the common vulnerabilities in terms of causes, implementation, and preventive measures. As shown in Table 1, we classify these vulnerabilities into three layers, including the blockchain layer, the virtual machine layer, and the solidity layer.

3.1 Timestamp Dependency

Timestamp is an important feature of the blockchain network. It can be treated as an anti-fake tag that can help miners identify whether a new block is valid. While packaging the transactions into a new block, the miner will apply a timestamp for that block [103]. Normally, the timestamp depends on the system time of the machine that mines the block. Although the block timestamp seems random, a 900-second timestamp tolerance is allowed in the Ethereum blockchain so that miners can set the timestamp within a short interval while it is legal. Thus, if the timestamp is used as a condition in a smart contract or to generate random numbers to perform important operations (such as Ether transfer), a malicious miner can bypass the validation and profit by slightly adjusting the timestamp. In a nutshell, the timestamp dependency vulnerability is a potential security issue brought about by using the *block.timestamp* as a condition to determine the execution of an operation in a smart contract. The prevention techniques for timestamp vulnerabilities include avoiding using *block.timestamp* to generate random numbers, or avoiding it being a key decision condition.

3.2 Transaction-ordering Dependency

Transaction-ordering dependency (TOD) is also known as race condition or front running. Front running refers to obtaining large transactions that may affect asset prices from non-public information in advance, to buy and sell first to obtain profits, which has long been a problem in traditional financial markets [60]. In the blockchain system, miners are responsible for selecting and packaging the transactions into blocks, and the block state depends on the transactions, which means that the block state is determined by the miners. The miners order and include transactions into new blocks according to rewards, the higher the *gasPrice* is, the faster the transaction will be processed. A contract may be vulnerable to Transaction-ordering attacks if the decision condition is based solely on transaction ordering.

For example, a puzzle contract will reward the first account with 100 Ethers for finding the answer. An attacker can observe all those who submit answers in the trading pool and verify whether the answers are correct. If the attacker finds a transaction with the correct answer, he can then submit another transaction including the correct answer with a higher *gasPrice*. According to Ethereum's gas mechanism, the attacker will get the reward, while


```

1 contract EtherBank {
2     uint256 public limitAmount = 1 ether;
3     mapping(address => uint256) public etherBalances;
4
5     function withdrawEtherAmount (uint256 _amountOfWei) public {
6         require(etherBalances[msg.sender] >= _amountOfWei);
7         // require withdraw amount less than limitAmount
8         require(_amountOfWei <= limitAmount);
9         require(msg.sender.call.value(_amountOfWei)());
10        etherBalances[msg.sender] -= _amountOfWei;
11    }
12
13    function depositEtherAmount() public payable {
14        etherBalances[msg.sender] += msg.value;
15    }
16 }

```

Fig. 1. Re-entrancy vulnerable contract.

the original first solver gets nothing. Transaction-ordering dependency vulnerability originates from the block packaging and gas price mechanisms. It can be avoided by some best development practices, such as setting the upper bound of the *gasPrice*, or hiding transaction information.

3.3 Reentrancy

Re-entrancy is a typical vulnerability of the Ethereum smart contract [53]. On the blockchain, some operations cannot be completed by one smart contract. At this time, calls between smart contracts are required. External calls are a feature of the Ethereum smart contract. Users can call and use smart contracts externally, and these calls usually include Ether sending operations. There is a fallback function in the smart contract that has no parameters and no return value. This fallback function will be triggered whenever the contract accepts the user's transfer and receives Ethers. If the fallback function contains the code calling itself, it will allow the user to re-enter the contract. These external calls can be hijacked by attackers and transfer money by continuously executing the withdrawal function in the fallback function. Some attackers use this vulnerability to steal Ethers in smart contracts. The DAO attack [59] is one of the most representative examples, which eventually led to the hard fork of Ethereum. The following example briefly illustrates how re-entrancy attacks happen.

As shown in Figure 1, there is a contract called *EtherBank*, which allows the depositor to withdraw up to 1 Ether. There is a public function *withdrawEtherAmount* to allow the sender to specify the amount of Ether to withdraw. When the sender's withdrawal amount passes a series of validations, the code in the ninth line will send the Ethers to the user, where the Re-entrancy attack might occur.

Figure 2 is an example of an attack contract. This contract calls the transfer function of the target contract and then re-enters the target contract itself through the fallback function for recursive calls. An attacker can instantiate a contract object by providing the address of *EtherBank* contract. The attacker will call *attack()* function to store a certain amount of Ether. Once the function is called, the following happens:

- attack.sol - line 11: The function *depositEtherAmount()* will be called. Currently, the attacker deposits one Ether into the *EtherBank* contract.
- attack.sol - line 13: Here, the attacker tries to withdraw one Ether from the *EtherBank* contract. Theoretically, the *EtherBank* contract will return one Ether to the attacker's account.
- attack.sol - line 18: Due to the fallback function, the *withdrawEtherAmount()* function will be continuously called until the victim's contract balance reaches zero.

The final result is that the attacker only deposits one Ether but can withdraw all Ethers from the contract account. To sum up, the main conditions for re-entrancy vulnerability include: (1) Ether transfer happens before the modification of the contract state variable; (2) the use of a CALL instruction for Ether transfer; and (3) the transfer target contract has a malicious fallback function. The re-entrancy vulnerability can be avoided by

```

1 contract Attack {
2     reEntrancyVulnerable public vulnerableContract;
3
4     // initialise with constructor
5     constructor(address _reEntrancyAddress) {
6         vulnerableContract = reEntrancyVulnerable(_reEntrancyAddress);
7     }
8
9     function attack() public payable {
10        require(msg.value >= 1 ether);
11        vulnerableContract.depositEtherAmount.value(1 ether)();
12        // the fallback function call
13        vulnerableContract.withdrawEtherAmount(1 ether);
14    }
15
16    // fallback function - where the magic happens
17    function () public {
18        vulnerableContract.withdrawEtherAmount(1 ether);
19    }
20 }
21

```

Fig. 2. Re-entrancy attack contract.

changing the order of Ether transfer and storage state modification. The official Solidity security guide also recommends using *send()* and avoiding *call.values* for Ether transfer.

3.4 Short Address

The vulnerability was first reported by the GÖLEM technical team in April 2017 [31]. The short address attack (also known as the parameter attack) refers to when the attacker makes a contract call by constructing an address with some zeros at the end of the address and deliberately removes the zeros in the call parameter, so as to shift and amplify the second parameter [98] by using the EVM data auto-completion mechanism. This operation allows attackers to abuse the transfer function to illegally obtain a larger amount of ERC20 tokens. Short address attacks usually occur in transactions (such as requesting a withdrawal on an exchange). The user initiates a contract transfer on the exchange, targeting a maliciously constructed short address. If the exchange does not verify the length of the user input, the actual transfer amount will increase sharply due to the short address vulnerability, resulting in a large amount of capital loss. The short address vulnerability is essentially caused by the underlying design flaw of the EVM. Effective preventive measures include adding verification of the length of the user input address in bits at the exchange level or adding a check mechanism for the address length at the code level. After this vulnerability was exposed in 2017, although the EVM layer has not resolved this problem, major exchanges have basically added address length checks, and the web3 layer has also added parameter format verification, so this kind of attack will not appear in the actual application scenarios.

3.5 Mishandled Exceptions

This vulnerability is also known as exception disorder or unchecked call return value [7]. Many operations in Ethereum smart contracts usually require interaction between contracts. The interaction between contracts can be carried out through two methods: (1) directly calling the function of the callee contract; (2) using external message calls (through *call*, *delegatecall*, *send* instructions). If the first method is used to directly reference the function of the callee contract, the exception can be handled correctly, and the contract will be terminated and reset to the state at the beginning of the call. However, when using an external message call, the exception message is not propagated back to the caller contract. If the exception is not handled correctly, the contract may continue to be executed and cause a capital loss. Multiple conditions can trigger exceptions, and this vulnerability can be classified into subvulnerabilities:

Gasless send: When using *send* to transfer Ether, the fallback method of the receiver has a gas limit of 2300 [75]. If the fallback function of the receiver contains many instructions with high gas consumption, an

out-of-gas exception will be triggered, indicating that there's not enough gas to complete the operation. If there is no exception handling mechanism, the contract may still believe that the send operation is successful and the balance will be updated, but the Ether is not actually received.

Call stack depth limit: Note that this problem has been resolved after the implementation of EIP-150 [15]. In the original EVM structure design, the maximum stack size was set to 1024 [94]. When one contract calls another, the depth of the call stack increases by 1. If the stack depth reaches the maximum limit, an exception will be thrown. According to Luu's research [54], the call stack depth of normal contract invocations will not exceed 50. However, an attacker can deliberately make the stack depth exceed the limit by filling the call stack in advance through multiple contract calls and transactions, resulting in contract call failure. If the exception is not handled correctly, the victim contract will continue to be executed, and the attacker can benefit from it. After the Ethereum Tangerine Whistle hard fork, the gas calculation method for external calls has been modified, and the call depth will never exceed 1,024 frames.

To sum up, the mishandled exception vulnerability refers to a series of vulnerabilities caused by the failure to explicitly check the return value (i.e., whether the call operation is successful) when using some low-level call methods for the invocations between contracts. Preventive techniques include the code that developers add to the caller contract to check the operation result, thereby circumventing state confusion that may arise from incongruous exception propagation. Although this exception propagation mechanism and its subclass call stack depth limit are EVM-related characteristics, since the essential cause of the vulnerability is bad programming practice, we classify it as a Solidity level vulnerability here.

3.6 Integer Overflow/Underflow

Integer errors are common errors in computer programming languages, including overflow and underflow [34]. Integer variables in computers have upper and lower bounds. If there is an out-of-bound error in arithmetic operations, two types of integer errors will occur. If the maximum range of the integer type is exceeded, the number will change from a maximum value to a minimum value or directly return to zero. This is called "overflow"; if the minimum range of the integer type is exceeded, the number will change from a small value or zero to a maximum value, which is called "underflow" [72]. In short, integer overflow means that the data stored in the memory unit storing integers is greater than the maximum value that the memory unit can store, which leads to overflow. In a smart contract, if the developer does not perform a safe overflow detection on the result of the operation, it will cause an integer overflow error. Integer overflows will cause errors in the operation of the smart contract program, allowing attackers to bypass checks and perform operations such as balance tampering, which may cause major losses.

In the Solidity language, the step size gap of each integer type is 8 bits, which supports uint8 to uint256, and int8 to int256. The largest unsigned integer type is uint256, and the maximum supported value is $2^{256} - 1$. When a uint256 variable reaches this value, adding 1 will bring it back to zero. Integer overflow is a very common type of security vulnerability in smart contracts, especially ERC20 token contracts. Attackers can use this vulnerability to pass in some specific values to skip some conditional verifications and make illegal and malicious requests (such as transferring a large amount of tokens). Some real-world cases include the BEC smart contract attack on Beauty Chain [12], and the SMT project attack in 2018. At present, in order to prevent this kind of attack, on the one hand, verification can be performed before and after the arithmetic logic. On the other hand, the SafeMath [69] library (including four safe methods, add, sub, mul and div), which is one of a set of smart contract libraries maintained by OpenZeppelin, can be used to perform the arithmetic logic safely.

3.7 Dangerous DelegateCall

In the Solidity language, both *call* and *delegatecall* are low-level functions used to facilitate interaction between smart contracts. Their key difference lies in that *delegatecall* only utilise the code from the target address, but the

```

1 * contract Wallet{
2 *     function() payable { //fallback function
3         if (msg.value > 0)
4             Deposit(msg.sender, msg.value);
5         else if (msg.data.length > 0)
6             _walletlibrary.delegatecall(msg.data) // Dangerous delegatecall!
7     }
8 }
9
10 * contract WalletLibrary {
11 *     function initWallet(address[] _owners, uint _required, uint _daylimit) {
12         initDaylimit(_daylimit);
13         initMultiowned(_owners, _required);
14     }
15 }

```

Fig. 3. Parity wallet code example.

execution context remains within the caller contract. Consequently, *delegatecall* is frequently employed in scenarios where multiple contracts invoke a library contract, thus avoiding redundant deployment and saving resources consumed during the deployment process. However, *delegatecall* is a dangerous function; by passing in specific parameters, the attacker can fully manipulate the current contract state (such as balance or storage variables).

Figure 3 shows a simple example. In the *Wallet* contract, line 6 calls *delegatecall* and passes the parameter *msg.data*, which allows the attacker to call any public function in the *WalletLibrary* contract. Therefore, the attacker can call *initWallet* in line 10 to become the owner of the wallet contract and can transfer Ether from the wallet contract to his own address. In July 2017, this vulnerability was revealed in the Parity Multisig wallet attack [70]. The attacker stole more than 150,000 Ethers from three high-security multi-signature contracts at a price of about 30 million U.S. dollars at that time. This vulnerability will not only allow attackers to steal user assets; if the attacker executes the kill command to remove a library contract code from the blockchain, other contracts using the library contract will not be able to work anymore, resulting in funds that cannot be transferred out and are frozen. In November of the same year, Github user “devops199” claimed to accidentally kill the Parity multi-signature library contract [101], resulting in the freezing of 500,000 ETH at a price of approximately US\$150 million.

3.8 Frozen Ether

This vulnerability is also called greedy contract, which is easier to understand. Because the Ether cannot be transferred out or the dependent external transfer function fails, the contract becomes greedy and leads to the loss of users’ funds [24]. An important role of smart contracts is to manage digital assets on a blockchain platform. Ethereum smart contracts typically use Ether as the digital asset to manage. Many contracts use digital asset transactions through Ether with users to implement high-value digital asset trade scenarios. Compared with normal accounts on Ethereum, contract accounts do not have private keys and can only manage the Ether in the accounts through contract codes (contract balance). Some contracts are used to accept and transfer funds to other addresses.

If the developer only codes for the function of receiving Ethers when developing the smart contract, but there is no function provided that allows Ethers to be transferred out or the function cannot be triggered, it will cause the Ethers stored in the contract to be permanently frozen. In addition, similar to 3.7, if the contract itself does not implement a transfer function by itself, it uses *delegatecall* to call some transfer functions in other contracts. Once these library contracts that provide the transfer function perform self-destruct operations, then the caller contracts that rely on these library contracts through *delegatecall* might have their balance frozen.

3.9 Tx.origin

Solidity has a global variable *tx.origin*, which traverses the entire call stack and returns the address of the account that originally sent the call (or transaction) [100]. Using this variable for authentication in a smart contract

makes the contract vulnerable to phishing-like attacks. The feasible defence measure is to avoid using *tx.origin* as much as possible. Not that you can never use *tx.origin* variables; there are reasonable usage scenarios for this variable. For example, if we want to control external contracts to call this contract, we can use some *require* verification statements to prevent some intermediate contracts from calling this contract. Besides, *msg.sender* is recommended to be used for permission control.

3.10 Denial of Service

Denial of service refers to the destruction of normal services through some malicious acts, which makes users unable to access or use services [76]. It is a common vulnerability in the traditional security field. In the smart contract, the access denial vulnerability will make the contract unable to complete the corresponding functions according to the expected design. Due to some characteristics of smart contracts (such as cannot update after deployment), this vulnerability has become more fatal in smart contracts. Once some contracts are attacked by denial of service, they will be permanently paralyzed and can no longer be called, which may lead to some Ether locked forever. There are many reasons that can cause denial of service, such as external call failure, gas reaching the block limit, unexpected self-destruction instructions, and so on. These reasons can usually be avoided through secure code specification.

3.11 Uninitialized Pointer

The EVM stores data either as storage or as memory, and it is possible to produce vulnerable contracts by inappropriately initialising variables. Local variables within functions are allocated to storage or memory, depending on their type. Uninitialised local storage variables can point to other unexpected storage variables in the contract, leading to intentional (i.e., the developer intentionally puts them there to attack later) or unintentional vulnerabilities [25]. The Solidity compiler raises uninitialised storage variables as warnings; thus, developers should pay careful attention to these warnings when building smart contracts. The developers should explicitly use the *memory* or *storage* keywords when dealing with complex types to ensure they behave as expected.

4 VULNERABILITY DETECTION TOOL

At present, the mainstream methods of automated vulnerability detection on the Ethereum smart contract platform include symbolic execution, formal verification, fuzzing, static analysis, machine learning, and so on. In this section, we introduce 14 representative tools in the current field, grouped into four categories according to the main techniques they employ, as shown in Table 2. At present, most vulnerability detection tools can only detect some vulnerabilities in a targeted manner, and there is no detection tool that can comprehensively analyze all security problems. Therefore, while introducing the principle of the tool, we also introduce the types of vulnerabilities that the tool supports to detect.

4.1 Symbolic Execution

Symbolic execution refers to the technique of using symbol value to represent the value of a program variable and then simulating the execution of the program to carry out correlation analysis under the condition that the program is not actually executed [10]. The main idea is to transform the uncertain inputs into symbolic values in the process of program execution so as to facilitate program execution and analysis and to explore the program branches as much as possible by constantly solving the path constraints.

Symbolic execution is a traditional automatic vulnerability detection technology that is also widely used in the smart contract field. When performing symbolic execution on a traditional program, problems such as path explosion might arise if the program code is too long. But smart contracts normally have less code, meaning fewer paths, which are better suited for symbolic analysis [81].

Table 2. Vulnerability Detection Tools

Applied Technology	Detection Tools	Type Analysis	Sequence analysis
Symbolic Execution	Oyente	bytecode	No
	Manticore	source code	Yes
	Osiris	bytecode	No
	Mythril	bytecode	Yes
	Maian	source code	No
Formal Verification	Zeus	source code	Yes
	Securify	source code	Yes
	Vandal	bytecode	No
Fuzzing	ContractFuzzer	bytecode	No
	Reguard	source code	Yes
	GasFuzzer	bytecode	No
	Harvey	source code	Yes
Others	Slither	source code	No
	SmartCheck	source code	No

4.1.1 Oyente. Luu et al. [54] investigated the security issues of Ethereum smart contracts and introduced a symbolic execution tool called Oyente, which aims at helping developers avoid writing or using smart contracts with security vulnerabilities. Developed by Python, Oyente is a static analysis tool based on symbolic execution technology that can detect vulnerabilities in the Ethereum platform. It supports four types of vulnerabilities: transaction-ordering dependence, timestamp dependence, mishandled exceptions and re-entrancy vulnerabilities. In addition, Oyente is one of the earliest research projects focusing on automated vulnerability detection for smart contracts; in many subsequent works, it serves as a simplified and scalable underlying symbolic execution engine for Ethereum smart contracts.

Oyente takes the contract bytecode and the current global state of Ethereum as input and symbolically executes the contracts at the Ethereum bytecode level to identify vulnerabilities. Oyente consists of four main modules: CFG Builder, Explorer, Core Analysis and Validator. Figure 4 shows the overview architecture of the Oyente tool. First of all, the CFG Builder performs a pre-analysis of the smart contracts, takes the basic execution block as the node and the execution jump as the edge, and then constructs the main control flow graph for each contract. However, not all jump relations can be completely determined in this process; further analysis is required. Then, the Explorer will conduct the symbolic execution of the smart contract to supplement the jump information. Essentially, the Explorer is an interpreter loop that sequentially executes the code of each basic block in the contract CFG. Every time it receives a state from the previous execution, it will execute an instruction. The Explorer uses the Z3 solver [102] to solve the conditional jump in the contract and determine which branch to analyse based on the satisfiability result. The core analysis module contains sub-components responsible for analysing smart contracts. For each supported vulnerability type, there are corresponding detection schemes. By comparing with the defined logic, the core analysis will discover the contract that is potentially vulnerable based on the information obtained from the explorer. Finally, the validator module verifies the analysis results, filters out false positives, and displays the final results to the user.

4.1.2 Manticore. Manticore [63] is an open-source symbolic execution tool provided by the Trail of Bits team. Manticore supports both the Ethereum platform and traditional computing environments; it is a dynamic symbolic execution framework that can be used to analyse both smart contracts and binaries. Manticore provides a user-friendly command-line interface and a Python-based programming API. Manticore compiles smart contract source code into bytecode as input for analysis and explores a large number of execution paths by replacing

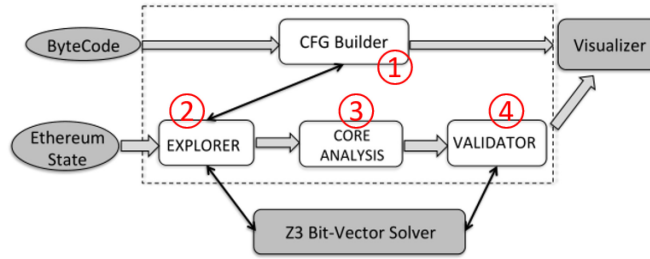


Fig. 4. Overview architecture of Oyente [54].

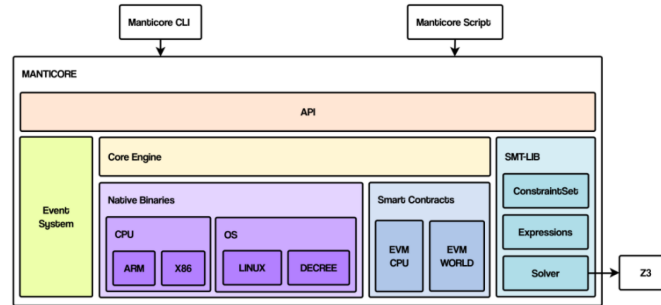


Fig. 5. Manticore overview [63].

program inputs with symbolic parameters. And with the help of an SMT solver, it can calculate the input that triggers the corresponding paths. The Trail of Bits team has used the Manticore tool in many code evaluation and program analysis projects. In addition to detecting vulnerabilities in smart contracts, Manticore has other features, such as exploring program paths, generating specific inputs, and instrumentation.

The main components of Manticore include the Core Engine, Ethereum, and Native Execution Modules in Figure 5. The Core Engine implements a platform-independent symbolic execution engine and proposes a state life cycle concept to manipulate and manage program states. The program state here refers to an abstract object that represents the state of the program at a certain point in execution. Manticore abstracts the execution process of the contract program into three states: ready, busy, and terminated, and these states can be converted to each other. The Core Engine supports parallelization and can be used to process multiple processes in the state queue. The Ethereum Execution Module is responsible for the symbolic execution of smart contracts. Manticore's Ethereum smart contract simulation environment supports tracking the status of multiple interactive contracts. The Native Execution Module is used for the symbolic execution of binaries. Besides, Manticore also has auxiliary modules, such as the **Satisfiability Module Theories (SMT-LIB)** module, which provides custom symbolic expression object models and enables SMT solving to calculate inputs.

4.1.3 Osiris. Torres et al. [87] developed the Osiris tool based on Oyente, which aims at detecting integer bugs. The Osiris tool combines symbolic execution and taint analysis techniques to accurately find security vulnerabilities caused by integer arithmetic errors in Ethereum smart contracts. Osiris supports both the solidity source code and bytecode of the smart contract as input, and the output of the analysis is whether the detected contracts contain any integer bugs. There are three types of integer bugs targeted by Osiris: arithmetic, truncation, and signedness. Arithmetic bugs include the common integer overflow and integer underflow. Truncation bugs refer to situations that may cause a loss of precision when trying to convert integer data types. Signedness

bugs happen when converting signed integer data to an unsigned one. In smart contracts, integer overflow is a very common vulnerability and can cause capital losses. However, it is impossible to accurately discover integer overflow vulnerabilities only through the corresponding incorrect computing instructions, because not all integer overflow errors are harmful. By combining taint analysis technology, Osiris is able to determine whether the discovered integer overflow errors are harmful, which can filter out many false positives. Osiris contains three main components: Symbolic Analysis, Taint Analysis, and Integer Error Detection. The Symbolic Analysis component builds the CFG, conducts the symbolic execution of different paths, and then passes the result of each instruction to the other two components. The Taint Analysis component is responsible for checking the taint in the stack, memory, and storage. The Integer Error Detection components check whether there are potential integer errors in the executed instructions.

4.1.4 Mythril. Mythril [64] is an open-source security analysis tool for Ethereum bytecode developed and maintained by ConsenSys. It combines symbolic execution, taint analysis, and SMT solving techniques to detect multiple types of vulnerabilities. Mythril officially provides 14 analysis modules and can detect vulnerabilities in categories such as re-entrancy, integer overflow, and so on. Unlike other tools that only support the Ethereum platform, Mythril also supports EVM-compatible blockchains such as Quorum [73], and TRON [89]. It provides a command-line tool written in Python for interactive analysis of smart contracts. Mythril takes EVM bytecode as input and uses concolic analysis to perform symbolic execution on specific execution paths. Mythril has a symbolic execution backend called Laser-Ethereum, which is a symbolic interpreter for EVM bytecode. Laser-Ethereum takes one or more smart contract accounts as input and returns a set of abstract program states. In addition, the ConsenSys team also provides a cloud-based security analysis service for smart contracts called MythX [77]. MythX has been integrated into many mainstream development frameworks, such as Truffle and Remix, and is capable of more comprehensive remote security analysis through simple and easy-to-use APIs.

4.1.5 Maian. The research of Nikolic et al. [67] focuses on a specific type of trace vulnerability and provides a symbolic analysis tool for Ethereum smart contract bytecode called Maian. There are mainly three types of trace vulnerabilities targeted by Maian, including prodigal, suicidal, and greedy. First, prodigal contracts refer to contracts that leak funds to any user. An attacker can hijack this kind of contract and then send funds from it to random users multiple times to gain profit. Suicidal contracts mean that if a contract can be killed by any account executing the SUICIDE instruction, it is considered a suicide contract. Many contracts usually leave a security backup option for being killed by the owner in emergency situations, such as being attacked or malfunctioning. But if any user can kill the contract, there will be serious security risks. For example, in the notorious Parity wallet incident, a GitHub user accidentally killed a library contract, resulting in the freezing of more than 500,000 Ethers. The third type is greedy contracts, which means that the contract can receive the user's funds transfer but is not able to send Ether out or send funds by invoking a third-party library contract while the library contract cannot be accessed.

In order to find these three trace vulnerabilities, Maian combines symbol analysis and concrete validation technology. As shown in the Figure 6, bytecode and analysis specifications are used as input to the symbolic analysis component. Analysis specifications include the targeted vulnerability type and the so-called invocation depth, that is, the depth of the search space. Maian implements a customized EVM that supports more than 120 instructions in the Ethereum stack language to promote symbolic contract execution until a trace that meets the predetermined attributes is found and flags the contract. Then, the concrete execution component of Maian is responsible for executing the contract to verify whether the result is correct. The developers implemented a test framework that supports the creation of private forks at any block height based on the Go-Ethereum toolkit so that the test results will not affect the contract state of the Ethereum main network. After testing nearly 1 million contracts, the evaluation results show that Maian can effectively mark contracts with trace vulnerabilities with a low false-positive rate.

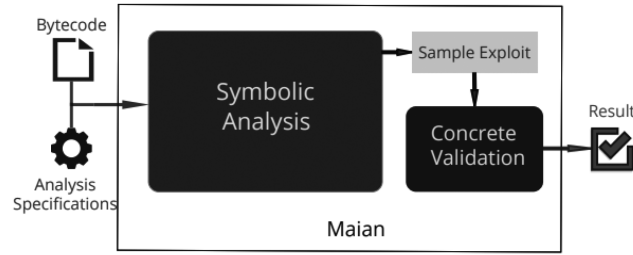


Fig. 6. Maian overview [67].

4.2 Formal Verification

Formal verification technology, in simple terms, is the use of mathematical methods to prove that the program is bug-free. Formalization refers to the description of the program through rigorous and verifiable description language or logic so that it can be subjected to rigorous mathematical reasoning and verification [35]. Formal technology is very common in the field of traditional hardware design, mainly due to the long hardware design cycle and high development costs. For example, when designing a CPU, formal verification technology will be used to ensure correctness. In the domain of formal verification in software, a complete and accurate formal description of program logic and instructions is a prerequisite for ensuring effectiveness. Automatic formal verification has three main types [28]: automatic theorem proving, model checking, and abstract interpretation. As for the formal verification of smart contracts, most of the current studies mainly focus on the formal description of smart contract instructions, and only a few studies pay attention to the security verification of the contract. Compared with other vulnerability detection technologies, the formal verification of smart contracts still faces many challenges.

4.2.1 Zeus. Sukrit et al. [45] studied and classified some known issues on smart contracts, provided the Zeus framework to ensure the correctness of safe programming practices for smart contract development, and verify the fairness of business logic. Zeus is an automated smart contract formal verification tool that supports the detection of six security vulnerabilities, including re-entrancy, unchecked send, failed send, integer overflow/underflow, block/transaction state dependency, and transaction ordering dependency. Zeus's formalism was inspired by Oyente, and conducted a comparative in the evaluation. The evaluation results on their dataset show that Zeus has improved its analysis speed and lowered the false positive rate.

Zeus is composed of three parts: Policy builder, Source code translator and Validator. Zeus takes Solidity source code and security policies written in a specific language as input. Firstly, the policy builder performs static analysis on the source code. In this step, Zeus leverages taint analysis technology to build the policy specification and extract predicates. The predicates are then inserted into the contract code as assertions. Then the source code translator converts the processed source code into LLVM bytecode. Zeus defines an abstract language to formalize Solidity semantics, which can capture the relevant structure of smart contract programs. At the same time, Zeus has constructed an XACML styled [96] five-tuple policy specification to formalize the policy by transforming the policy in the code into the assertions in the abstract language. By using the LLVM bytecode, Zeus can separate the implementation of translation and verification parts, and combine them with some mature formal verification solutions in the industry. Meanwhile, it enables the support of the smart contract on different blockchain platforms written in high-level languages such as GO and Java. Finally, the validator uses constrained horn clauses to verify and determine assertion conflicts to identify security vulnerabilities.

4.2.2 Securify. Securify [90] is an automatic and scalable security verification tool for Ethereum smart contracts. It takes EVM bytecode and so-called security properties as input (note that it also supports Solidity source code as input and will compile the source code into bytecode in advance). The security properties are pre-defined

```

contract OwnableWallet {
    address owner;

    // called by the constructor
    function initWallet(address _owner) {
        owner = _owner; // any user can change owner
        // more setup
    }

    // function that allows the owner to withdraw ether
    function withdraw(uint _amount) {
        if (msg.sender == owner) {
            owner.transfer(_amount);
        }
    }
    // ...
}

```

Fig. 7. Frozen ether example.

compliance and violation patterns expressed in a domain-specific language. Users can write patterns by themselves, which makes Securify scalable. The design of Securify is based on pattern matching. With given properties, it can detect different types of vulnerabilities in the smart contracts, such as re-entrancy, delegate-call, and frozen ether. The output of the analysis is the location of the specific vulnerability.

Securify's analysis consists of three steps. First, in the decompiling process, the stack-oriented bytecode will be converted into a **static-single assignment (SSA)** form, which excludes all stack operations. And the compiler will build the CFG based on the decompiled instructions. Securify then analyses the contract to infer semantic facts and describes the semantic facts in Datalog syntax. This step is fully automated with the help of the existing tool Souffle [43]. The inferred semantic facts can abstractly describe data dependence and control dependence. Finally, Securify checks the input security patterns, which are described by its DSL. By checking the semantic facts and the matching of the two kinds of patterns, it can determine whether the given security properties are satisfied, thereby analysing the security of the smart contract. Securify developers used two datasets (bytecode and Solidity) to evaluate the performance. The results show that, compared with other advanced tools such as Oyente and Mythril, Securify can accurately identify most vulnerabilities.

At the beginning of 2020, the developers released the latest version of Securify, v2.0. In the new version, Securify2 directly analyzes Solidity source code instead of EVM bytecode, which is the biggest difference from the first version. Although it is more difficult to analyze the source code, the richer language structure contains more important semantic information, such as the storage model of the contract, which does not exist in the bytecode. Securify2 introduces an IR based on the MLton compiler to handle complex Solidity source code. At the same time, through fully declarative analysis, Securify2 significantly improves analysis speed and scalability. The new version of Securify is completely open-source, able to analyze contracts written in Solidity ≥ 0.58 , and currently supports 37 vulnerability patterns.

4.2.3 Vandal. Lexi et al. [14] developed Vandal, a security verification framework for Ethereum smart contracts based on static analysis. Similar to Securify, Vandal uses the Souffle tool to interpret semantic information in the Datalog language. Vandal takes EVM bytecode as input, and the analysis process consists of two parts. First, Vandal has an analysis pipeline that converts low-level EVM bytecode into logical relationships. This pipeline in Figure 8 has five main parts. The scraper extracts the EVM bytecode. The disassembler converts the bytecode

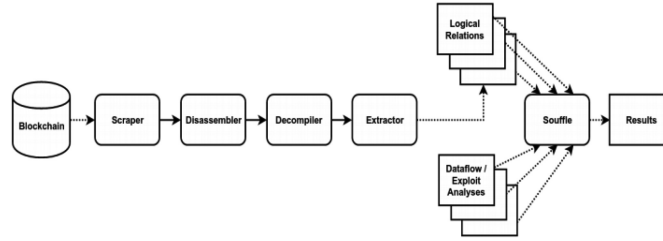


Fig. 8. Vandal framework overview [14].

into disassembly mode. The decompiler converts the stack-based bytecode into register-oriented IR. The extractor is based on the register transfer language; it generates logical semantic relations, including data and control flow dependencies. Then, the security analysis module uses a novel logic-driven method to identify the vulnerabilities in the smart contracts. Vandal supports many kinds of vulnerabilities, such as re-entrancy, unprotected suicide, use of tx.origin. Vandal’s developers used a dataset of 191 K contracts to conduct large-scale evaluations. The results show that Vandal is superior to Oyente, Mythril, and other tools in terms of analysis speed and false positive rate.

4.3 Fuzzing

Fuzzing is a popular and efficient software analysis technique. The idea of fuzzing was proposed by Miller et al. in the 1990s [61]. Its core idea is to automatically generate random test inputs for the program, and monitor abnormal program behaviours during execution, such as crashes, to discover potential program errors.

There are two main ways the fuzzer generates test cases [50], mutation-based and generation-based. Mutation-based is to mutate the original inputs by changing certain fields of the sample data, thereby generating new mutation inputs, which is a relatively simple way. The generation-based method needs to analyse the target program first and understand the relevant knowledge of the program input, such as format, protocol, or interface specifications. As a result, the inputs generated in this way are more easily passed by the program verification, and they are more likely to improve the coverage of the test.

According to the level of understanding of the program’s internal structure, fuzzing can be classified into three modes, black-box, white-box, and grey-box [50]. The black-box fuzzing does not need the internal analysis of the program, only triggers crashes through random inputs. White-box fuzzing means that the source code of the target program is available, so it can combine other program analysis techniques to improve coverage and vulnerability detection capabilities. The grey-box fuzzing is similar to the black-box fuzzing, but the result of the program execution can be leveraged as feedback.

In recent years, fuzzing has undergone extensive development in the field of software testing and is often used to detect security vulnerabilities in the software of computer systems. **American fuzzy lop (AFL)** [3] is a very popular open-source grey-box security-oriented fuzzer, developed by the former Google security expert Michal, Zalewski, and first released in 2013. AFL can fuzz the C/C++ programs running on Linux and has found hundreds of vulnerabilities in software, system kernels, and libraries. In addition, there are many related research projects and tools, such as LibFuzzer [80], CollAFL [29], and Hawkeye [19].

4.3.1 ContractFuzzer. Bo Jiang et al. [40] presented a novel fuzzing framework called ContractFuzzer for finding security vulnerabilities in Ethereum smart contracts. ContractFuzzer is the first black-box fuzzer for the Ethereum platform, and it can identify seven types of Ethereum smart contract vulnerabilities. For each supported vulnerability type, ContractFuzzer defines a test oracle for precise detection. ContractFuzzer has two components: an offline EVM instrumentation tool and an online fuzzer. The instrumented EVM can inspect the execution behaviour of smart contracts and obtain useful information for further analysis.

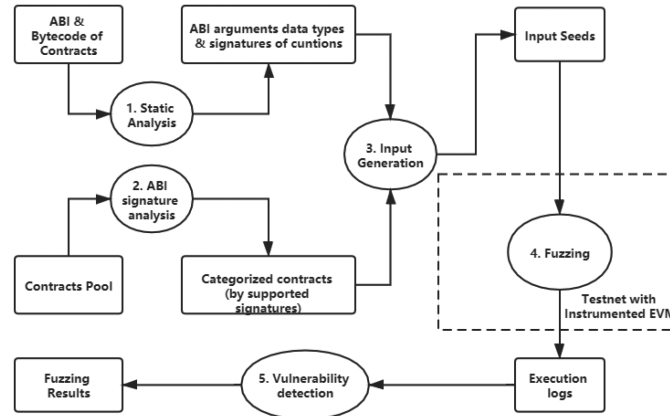


Fig. 9. Overview of ContractFuzzer.

Figure 9 shows the whole workflow of the ContractFuzzer tool. Firstly, ContractFuzzer conducts a static analysis of the ABI interface and bytecode of the contracts and extracts the data types of ABI function parameters and ABI signatures. Then it performs an ABI signature analysis of all contracts and indexes them according to the signature they support. Based on the results obtained from the first two analyses, ContractFuzzer can generate the input fuzzing seeds that meet the ABI specifications. Then the fuzzing test starts by randomly calling the corresponding ABI interface with the generated inputs. Finally, it analyses the execution logs to identify security vulnerabilities.

4.3.2 ReGuard. In Liu et al.'s work [53], they proposed a tool called ReGuard, which is built on fuzzing technology and specifically designed to dynamically detect the reentrancy vulnerabilities in the Ethereum smart contract. ReGuard has a translation-based framework that can translate Ethereum smart contracts written in Solidity to C++ programs. By extending some mature fuzzing engines, ReGuard generates random transactions, analyses the execution traces of C++ programs and uses a reentrancy automata to discover potential bugs.

The ReGuard tool is mainly composed of three key modules: Contract Transformer, Fuzzing Engine and Core Detector. It can take the source code or bytecode of the smart contracts as input. Initially, the input will be parsed into an intermediate representation. The source code will be parsed into an **abstract syntax tree (AST)**, and correspondingly, a **control flow graph (CFG)** is used as the IR of the bytecode. Then, the Contract Transformer transforms the IR into C++ code, which is semantically equivalent to the original code and can be used by the fuzzing engine to generate transactions. Meanwhile, the fuzzing engine embedded in ReGuard iteratively generates random transactions that cover different execution scenarios. By doing so, it can explore the different states of the contracts as much as possible to discover potential errors. ReGuard runs the C++ programs on the transactions with the combined runtime library and, during this process, investigates the execution traces of some relevant operations (e.g., function calls and returns). Finally, the reentrancy vulnerability identification process is conducted in the code detector. ReGuard compares the execution traces to the reentrancy automata to analyse and report bugs.

4.3.3 GasFuzzer. GasFuzzer [7] is another tool that uses fuzzing on the Ethereum platform to find security vulnerabilities. GasFuzzer is built on ContractFuzzer and proposes a novel idea to find gas-oriented exception vulnerabilities by manipulating the gas allowance of the contract. The evaluation result shows that compared with ContractFuzzer, GasFuzzer can detect 28% more exceptions disorder vulnerabilities. GasFuzzer has two mutation strategies to improve the effectiveness of vulnerability detection: gas-greedy and gas-levelling.

In the gas-greedy strategy, GasFuzzer will preferentially choose transactions with higher gas consumption as mutation inputs. In the EVM gas architecture, if a transaction consumes more gas, more opcodes and more important blockchain-related operations might be executed. Initially, GasFuzzer uses a similar method as ContractFuzzer to generate random transactions. These transactions are then mutated (GasFuzzer mutates the transactions by changing the input parameters) and sent to the blockchain for execution. Meanwhile, their gas consumption information will be collected. If these newly generated transactions consume more gas, they will be added to the input queue and wait for further mutations. GasFuzzer will continue to execute this process in a loop until the testing time runs out. Finally, the execution log is analysed to detect security vulnerabilities. In this strategy, GasFuzzer can increase the diversity of seed transactions through this input prioritization schema.

In the gas-levelling strategy, GasFuzzer mutates the gas allowance of the selected gas-expensive transactions. The seed poll is composed of gas-expensive transactions (transactions consuming the most gas) generated by the gas-greedy strategy. These transactions form a transaction sequence. GasFuzzer will select transactions from this sequence and mutate their gas allowance to generate a new transaction sequence. In this process, the gas consumption of these transactions is divided into several intervals, and then GasFuzzer mutates the gas allowance to make the mutated result fall into a randomly selected interval. At the same time, GasFuzzer introduces a novel gas coverage criterion, which works as the termination of the fuzzing test. GasFuzzer will repeat this process until the coverage of a set of gas allowance intervals for these selected transactions meets the threshold defined in advance.

4.3.4 Harvey. Harvey [95] is a grey-box fuzzer for smart contracts that have been used in industry. The developers have identified two challenges to applying grey-box fuzzing technology to smart contract analysis and proposed corresponding approaches to address the challenges.

Firstly, the grey-box fuzzing technology does not need complicated analysis steps such as program analysis or constraint solving. It only requires lightweight instrumentation, which balances the effectiveness and speed of the analysis. But still, it generates the test inputs randomly, so there are still a large number of inputs to execute the same path, and a high coverage rate cannot be guaranteed. Harvey applies an input prediction technology to determine whether the generated random inputs are conducive to exploring new paths by instrumenting before the conditional statements in the smart contracts rather than performing arbitrary mutations, thereby improving the performance and effectiveness of the analysis.

Secondly, there are many different states in the life cycle of a smart contract, and the fuzzer needs to generate a call sequence to explore various possible states. But as the sequence gets longer, the number of possible sequences increases exponentially. Therefore, Harvey has introduced a demand-driven sequence generation technology to generate simple call sequences through the read and write dependencies of global variables between different functions to improve the coverage and effectiveness of fuzzing.

4.4 Others

In Ethereum smart contract vulnerability detection, in addition to the above-mentioned symbolic execution, formal verification, and fuzzing, there are some other techniques such as static program analysis and machine learning. In the field of automated vulnerability detection for traditional software, static program analysis has always been a mainstream method. The main feature of static analysis is to scan and analyze the code in various ways without executing the actual program to verify whether the code is reliable and compliant and to find security vulnerabilities in the code [32]. Common static analysis methods include control flow analysis, taint analysis, abstract syntax tree analysis, and so on. Because there is no need to run the program, static program analysis tools often have high efficiency, but at the same time, they may also face the problem of a relatively high false positive rate. In this section, we focus on two static analysis tools. In addition to this, recent studies have also applied machine learning techniques to the practice of smart contract vulnerability detection, such as ContractWard [92], ILF [36], and Soliaudit [52].

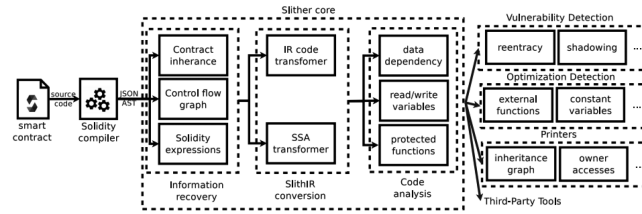


Fig. 10. Slither framework overview.

4.4.1 Slither. Slither is a smart contract static analysis framework [27] that can automatically detect vulnerabilities in smart contracts and code optimizations missed by the compiler. Slither now supports the detection of more than 80 types of smart contract exceptions, including a variety of common security vulnerabilities such as re-entrancy. Slither can be integrated with some common frameworks to analyze a smart contract. Slither also provides a printer module that can summarize and display contract information to help users study the contract code. The information that users can obtain through the printer module includes various graph-based representations (such as the CFG, inheritance graph, and call graph) and an easy-to-understand information summary (such as the code quality report and the number of problems found). In addition, Slither provides interactive Python APIs. The users can write custom tools or scripts to perform higher-level code review or analysis to meet specific needs.

Slither's analysis process is mainly divided into the following steps: The smart contract source code is compiled by *solc* to obtain the Solidity AST as the input of Slither. Slither will first obtain the inheritance graph and control flow graph of the contract, as well as the list of functions in the contract. Then, Slither converts the contract code into its own intermediate representation language called SlithIR. SlithIR uses a static single assignment form to facilitate various code analyses. Then comes the code analysis stage, where Slither will perform some pre-defined computations, including the dependencies between the functions and various variables in the contract, the read and write of variables, and the permission control of functions. Finally, the calculation results that contain the enhanced information will be fed into other modules for vulnerability detection, optimized detection, and information printers. The whole overview of Slither is shown in Figure 10.

One of the main contributions of the Slither tool is to propose an intermediate representation language called SlithIR. The instruction set of SlithIR contains more than 30 different instructions. The instructions are converted from Solidity expressions and associated with the nodes on the previously obtained control flow graph. SlithIR uses a static single assignment form (SSA), which is a commonly used form for static analysis. The main advantage is that it can simplify the analysis of data dependencies. There are other different intermediate representations for smart contracts, such as Scilla [78] and IELE [46]. Scilla is the IR used in the Zilliqa blockchain, based on the concept of communicating automata, but it is not yet certain that Scilla can support the full range of Solidity languages. IELE is the IR provided by the Cardona blockchain [17]. Different from SlithIR, IELE uses the reads and stores of the storage memory to represent state variables (while SlithIR just uses variables), which makes the analysis more complicated. However, there is still a lack of formal semantics and low-level information (such as gas consumption) in SlithIR, and it still has aspects that can be improved.

Slither's team evaluated Slither's performance from three aspects: vulnerability detection, code optimization detection, and source code understanding, and compared it with some state-of-the-art tools in the industry. In terms of vulnerability detection ability, the developers conducted two experiments. The first experiment used a dataset of more than 1,000 contracts with a large transaction volume obtained from Etherscan [26], and the other used two well-known contractsm DAO and SpankChain. There are three tools for simultaneous evaluation and comparison with Slither: SmartCheck [85], Solhint [82], and Securify [90]. The performance of the tool is evaluated by three indicators: analysis speed, robustness, and accuracy. Slither's analysis speed is the fastest

among the four tools, and there is no timeout. As for robustness, only 0.1% of Slither's contracts cannot be analyzed properly, which is far superior to SmartCheck and Securify. At the same time, Slither has the best accuracy and the lowest false positive rate. The results show that Slither can perform fast analysis while ensuring reliability and accuracy. In addition, in the evaluation of the other two functions, the results also show that Slither can be effectively used to detect optimizations in real-world code and help users improve their code understanding. Written in Python, Slither is open-sourced and available at [79].

4.4.2 SmartCheck. SmartCheck [85] is a static analysis tool for Ethereum smart contracts developed by Tikhomirov et al. and supports the identification of many types of security vulnerabilities. SmartCheck works at the source code level of the smart contract. It first performs syntactical and lexical analysis on the Solidity source code and then uses the ANTLR parse generator [6] and Solidity custom grammar to generate an XML tree as an **intermediate representation (IR)** to describe the analyzed abstract syntax results. On this basis, SmartCheck uses XPath queries on IR to check the security properties of the smart contract and discover vulnerabilities. Currently, SmartCheck supports multiple types of vulnerability detection, such as re-entrancy, timestamp dependency, denial-of-service, frozen ether, and so on.

SmartCheck completely converts the analyzed code into IR, and all elements in the code can be obtained through XPath matching. The line number information of the code is saved in the XML, which can help locate the positions of the problems found in the source code. In addition, SmartCheck can also support smart contracts written in other languages by adding specified syntax and database extensions. During the evaluation process, the developers compared SmartCheck with three other static detection tools, Oyente [54], REMIX [74] and Securify [90], and the results showed that this tool is an efficient analysis tool and that code issues have been detected in most contracts.

5 CHALLENGES

In this section, we will focus on the advantages and disadvantages of three mainstream Ethereum smart contract vulnerability detection techniques: symbolic execution, formal verification, and fuzzing. Additionally, we highlight the challenges faced by these techniques and identify corresponding research directions.

5.1 Challenges in Symbol Execution

Oyente is the first to implement a symbol execution engine for smart contracts, which is one of the earliest automated vulnerability detection efforts for smart contracts. After that, Manticore, Osiris, Mythril, Maianm and other tools using symbolic execution techniques began to appear. Symbolic execution-based tools mainly simulate real contract execution by increasing the transaction depth of execution analysis. These tools, combined with other techniques such as taint analysis, explore deeper state spaces and improve the accuracy of vulnerability detection by reducing false positive rates. However, path explosion is still a major problem, and how to extract key paths and reduce the path space may be a research direction in the current field.

For path exploration problems, utilizing sound analysis techniques [11] can be a possible solution to reduce the complexity. In other words, statically merging paths before feeding the solver. Although this approach works in many cases, it transfers the complexity to the solver, which leads to another challenge, i.e., the complexity of constraint solving, which is also worthy of further exploration.

The precision with which program statements are converted into symbolic constraints has a great impact on the coverage of symbolic execution. For example, memory modelling that approximates a fixed-digit integer variable with a concrete value can be very efficient, but on the other hand, it can also lead to imprecision, such as losing some symbolic execution paths or exploring some useless solutions. Precision and scalability are tradeoffs when overcoming the memory modelling issue. In the future, we should pay more attention to two aspects: (a) whether the code is high-level application code or low-level system code, and (b) the actual effects of different constraint solvers.

5.2 Challenges in Formal Verification

Formal verification techniques for smart contract vulnerability detection have also undergone significant development and maturity. Securify [90] is a relatively representative research in the early stages that mainly focuses on how to apply general formal modelling to the instructions of smart contracts. In addition, Zeus [45], VerX [71], and other research schemes proposed more effective security attribute verification methods for smart contracts to improve the accuracy of vulnerability detection and reduce manual participation in the detection process.

Despite the efficacy of formal verification in detecting smart contract vulnerabilities, widespread deployment remains problematic. Future research should modify verification specification descriptions for various business objectives, overcome technical constraints such as high cost and inability to adapt to large-scale contracts, and increase the application breadth of formal verification. For example, for some complex designs, it is impossible to define all specifications with one attribute, and the high cost of computing resources must also be considered.

5.3 Challenges in Fuzzing

As a dynamic software analysis technology, fuzzing has certain effectiveness when it is used in vulnerability detection in smart contracts. But at the same time, it faces the challenges of low accuracy and low code coverage. At present, the research on applying fuzzing to smart contract vulnerability detection is still in the initial stages. The degree of automation is rather low, and most fuzzing-based tools have many application limitations. For instance, Contractfuzzer [40] is difficult to find all potential errors due to the randomness of test case generation, the limited system behaviours, and the unachievable ideal path coverage. We summarize two challenges in applying fuzzing to smart contracts for future work.

First, improving the ability to identify more types of vulnerabilities. The current work [41] is mainly to find injection points (crash places), while missing the detection of other non-crash type vulnerabilities. Research on side channel vulnerabilities [68] is a hot topic at present. New technologies that can automatically detect privilege escalation, remote code execution, and other important security vulnerabilities need to be studied.

Second, improving the ability to detect complex and evasive vulnerabilities. The triggering conditions of some vulnerabilities are very complex, and some detection methods are quite time-consuming and labor-intensive. Structure-aware fuzzing [55], grammar-based fuzzing [30], and grey-box fuzzing [38] that integrate static analysis and symbolic execution might be good choices. Meanwhile, software is always changing, so techniques that can target software patches will prove essential for finding vulnerabilities. There is a need to facilitate vulnerability finding like AFLFast [13] does, and it is necessary to study how to use the parallelization function of the GPU and other components to improve the efficiency of execution. In addition, it is also important to rank vulnerabilities according to their importance.

6 CONCLUSION

With the rapid development of blockchain technology, its application has been widely spread across various fields. As one of the most representative applications of blockchain technology, smart contracts currently face several vulnerabilities, causing some security incidents and substantial economic losses. The security issue of smart contracts has become a major issue in blockchain security research. We conducted a systematic investigation into the security of the Ethereum smart contract. First, we introduce the background knowledge of blockchain technology and the related features of the Ethereum smart contract. For the security vulnerabilities of Ethereum smart contracts, we summarize 11 common vulnerabilities from the three-layer threat model of the blockchain layer, the Ethereum virtual machine layer, and the solidity language layer and analyze the causes, exploit methods, and effective solutions of the vulnerabilities. We then introduce 14 mainstream vulnerability detection tools according to their main techniques, such as symbolic execution, formal verification, and fuzzing. We describe the main principles of these tools and the categories of vulnerabilities they can detect. Finally, we compare

these introduced Ethereum smart contract vulnerability detection techniques and introduce their advantages and disadvantages. Open challenges and research directions regarding these vulnerability detection techniques are discussed as well.

REFERENCES

- [1] Accessible: Retrieved from <https://github.com/pirapira/bamboo>. Bamboo Language.
- [2] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. 2021. Uniswap v3 Core.
- [3] AFL. 2018. *AFL - American Fuzzy Lop*. Retrieved June 20, 2022 from <http://lcamtuf.coredump.cx/afl/>
- [4] Elvira Albert, Jesús Correas, Pablo Gordillo, Guillermo Román-Díez, and Albert Rubio. 2020. GASOL: Gas analysis and optimization for ethereum smart contracts. In *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 118–125.
- [5] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. 2018. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the 13th EuroSys Conference*. 1–15.
- [6] ANTLR. 2018. *ANTLR - Another Tool for Language Recognition*. Retrieved June 20, 2022 from <http://www.antlr.org/>
- [7] Imran Ashraf, Xiaoxue Ma, Bo Jiang, and WK Chan. 2020. GasFuzzer: Fuzzing ethereum smart contract binaries to expose gas-oriented exception security vulnerabilities. *IEEE Access* 8 (2020), 99552–99564.
- [8] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. 2017. A survey of attacks on ethereum smart contracts (sok). In *Proceedings of the International Conference on Principles of Security and Trust*. Springer, 164–186.
- [9] Lennart Bader, Jens Christoph Bürger, Roman Matzutt, and Klaus Wehrle. 2018. Smart contract-based car insurance policies. In *Proceedings of the 2018 IEEE Globecom Workshops*. IEEE, 1–7.
- [10] Roberto Baldoni, Emilio Coppa, Daniele Cono D’elia, Camil Demetrescu, and Irene Finocchi. 2018. A survey of symbolic execution techniques. *ACM Computing Surveys* 51, 3 (2018), 1–39.
- [11] Josh Berdine, Cristiano Calcagno, and Peter W O’hearn. 2005. Symbolic execution with separation logic. In *Proceedings of the Asian Symposium on Programming Languages and Systems*. Springer, 52–68.
- [12] Mueller Bernhard. 2018. *A Disastrous Vulnerability Found in Smart Contracts of BeautyChain (BEC)*. Retrieved June 20, 2022 from <https://medium.com/secbit-media/a-disastrous-vulnerability-found-in-smart-contracts-of-beautychain-bec-dbf24ddb30e>
- [13] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2017. Coverage-based greybox fuzzing as markov chain. *IEEE Transactions on Software Engineering* 45, 5 (2017), 489–506.
- [14] Lexi Brent, Anton Jurisevic, Michael Kong, Eric Liu, Francois Gauthier, Vincent Gramoli, Ralph Holz, and Bernhard Scholz. 2018. Vandal: A scalable security analysis framework for smart contracts. arXiv:1809.03981. Retrieved from <https://arxiv.org/abs/1809.03981>
- [15] Vitalik Buterin. 2017. *EIP-150: Gas Cost Changes for IO-heavy Operations*. Retrieved June 20, 2022 from <https://eips.ethereum.org/EIPS/eip-150>
- [16] Vitalik Buterin et al. 2013. Ethereum white paper. *GitHub Repository* 1 (2013), 22–23.
- [17] Cardano. 2019. *Cardano Blockchain Platform*. Retrieved June 20, 2022 from <https://docs.cardano.org/en/latest/>
- [18] Huashan Chen, Marcus Pendleton, Laurent Njilla, and Shouhuai Xu. 2020. A survey on ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Computing Surveys* 53, 3 (2020), 1–43.
- [19] Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2095–2108.
- [20] Jiachi Chen, Xin Xia, David Lo, and John Grundy. 2021. Why do smart contracts self-destruct? Investigating the selfdestruct function on ethereum. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2021), 1–37.
- [21] Silvia Crafa, Matteo Di Pirro, and Elena Zucca. 2019. Is solidity solid enough?. In *Proceedings of the International Conference on Financial Cryptography and Data Security*. Springer, 138–153.
- [22] Chris Dannen. 2017. *Introducing Ethereum and Solidity*. Vol. 1. Springer.
- [23] Megan DeMatteo. 2022. *Bitcoin Price History: 2009 to 2022*.
- [24] Giuseppe Destefanis, Michele Marchesi, Marco Ortu, Roberto Tonelli, Andrea Bracciali, and Robert Hierons. 2018. Smart contracts vulnerabilities: A call for blockchain software engineering?. In *Proceedings of the 2018 International Workshop on Blockchain Oriented Software Engineering*. IEEE, 19–25.
- [25] Wesley Dingman, Aviel Cohen, Nick Ferrara, Adam Lynch, Patrick Jasinski, Paul E. Black, and Lin Deng. 2019. Defects and vulnerabilities in smart contracts, a classification using the NIST bugs framework. *International Journal of Networked and Distributed Computing* 7, 3 (2019), 121–132.
- [26] Etherscan. 2021. Etherscan—the ethereum blockchain explorer. <https://etherscan.io/> (2021).
- [27] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: A static analysis framework for smart contracts. In *Proceedings of the 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain*. IEEE, 8–15.

- [28] Xiaotao Feng, Qin Wang, Xiaogang Zhu, and Sheng Wen. 2019. Bug searching in smart contract. arXiv:190500799. Retrieved from <https://arxiv.org/abs/1905.00799>
- [29] Shuitao Gan, Chao Zhang, Xiaojun Qin, Xuwen Tu, Kang Li, Zhongyu Pei, and Zuoning Chen. 2018. Collafl: Path sensitive fuzzing. In *Proceedings of the 2018 IEEE Symposium on Security and Privacy*. IEEE, 679–696.
- [30] Patrice Godefroid, Adam Kiezun, and Michael Y Levin. 2008. Grammar-based whitebox fuzzing. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 206–215.
- [31] Project Golem. 2017. *How to Find \$10M Just by Reading the Blockchain*. Retrieved June 20, 2022 from <https://blog.golempoint.net/how-to-find-10m-just-by-reading-the-blockchain/>
- [32] Anjana Gosain and Ganga Sharma. 2015. Static analysis: A survey of techniques and tools. In *Proceedings of the Intelligent Computing and Applications*. Springer, 581–591.
- [33] Neville Grech, Michael Kong, Anton Jurisevic, Lexi Brent, Bernhard Scholz, and Yannis Smaragdakis. 2020. MadMax: Analyzing the out-of-gas world of smart contracts. *Communication of the ACM* 63, 10 (2020), 87–95.
- [34] Christopher G. Harris. 2019. The risks and challenges of implementing ethereum smart contracts. In *Proceedings of the 2019 IEEE International Conference on Blockchain and Cryptocurrency*. IEEE, 104–107.
- [35] Osman Hasan and Sofiene Tahar. 2015. Formal verification methods. In *Proceedings of the Encyclopedia of Information Science and Technology, Third Edition*. IGI Global, 7162–7170.
- [36] Jingxuan He, Mislav Balunović, Nodar Ambroladze, Petar Tsankov, and Martin Vechev. 2019. Learning to fuzz from symbolic execution with application to smart contracts. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 531–548.
- [37] Seyoung Huh, Sangrae Cho, and Soohyung Kim. 2017. Managing IoT devices using blockchain platform. In *Proceedings of the 2017 19th International Conference on Advanced Communication Technology*. IEEE, 464–467.
- [38] Antonio Ken Iannillo, Roberto Natella, Domenico Cotroneo, and Cristina Nita-Rotaru. 2017. Chizpurfle: A gray-box android fuzzer for vendor service customizations. In *Proceedings of the 2017 IEEE 28th International Symposium on Software Reliability Engineering*. IEEE, 1–11.
- [39] Faisal Jamil, Lei Hang, KyuHyung Kim, and DoHyeun Kim. 2019. A novel medical blockchain model for drug supply chain integrity management in a smart hospital. *Electronics* 8, 5 (2019), 505.
- [40] Bo Jiang, Ye Liu, and WK Chan. 2018. Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 2018 33rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 259–269.
- [41] Zu-Ming Jiang, Jia-Ju Bai, Kangjie Lu, and Shi-Min Hu. 2020. Fuzzing error handling code using {Context-Sensitive} software fault injection. In *Proceedings of the 29th USENIX Security Symposium*. 2595–2612.
- [42] Jiao Jiao, Shuanglong Kan, Shang-Wei Lin, David Sanan, Yang Liu, and Jun Sun. 2020. Semantic understanding of smart contracts: Executable operational semantics of solidity. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy*. IEEE, 1695–1712.
- [43] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On synthesis of program analyzers. In *Proceedings of the International Conference on Computer Aided Verification*. Springer, 422–430.
- [44] Mudabbir Kaleem, Anastasia Mavridou, and Aron Laszka. 2020. Vyper: A security comparison with solidity based on common vulnerabilities. In *Proceedings of the 2020 2nd Conference on Blockchain Research and Applications for Innovative Networks and Services*. IEEE, 107–111.
- [45] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. 2018. ZEUS: Analyzing safety of smart contracts.. In *Proceedings of the Ndss*. 1–12.
- [46] Theodoros Kasampalis, Dwight Guth, Brandon Moore, Traian Serbanuta, Virgil Serbanuta, Daniele Filaretti, Grigore Rosu, and Ralph Johnson. 2018. *Iele: An intermediate-level Blockchain Language Designed and Implemented using Formal Semantics*. Technical Report.
- [47] Zulfiqar Ali Khan and Akbar Siami Namin. 2020. A survey on vulnerabilities of ethereum smart contracts. arXiv:2012.14481. Retrieved from <https://arxiv.org/abs/2012.14481>
- [48] Minjeong Kim, Yujin Kwon, and Yongdae Kim. 2019. Is stellar as secure as you think?. In *Proceedings of the 2019 IEEE European Symposium on Security and Privacy Workshops*. IEEE, 377–385.
- [49] Sunny King and Scott Nadal. 2012. Ppcoin: Peer-to-peer crypto-currency with proof-of-stake. *self-published paper, August 19*, 1 (2012).
- [50] Jun Li, Bodong Zhao, and Chao Zhang. 2018. Fuzzing: A survey. *Cybersecurity* 1, 1 (2018), 1–13.
- [51] Xiaoqi Li, Peng Jiang, Ting Chen, Xiapu Luo, and Qiaoyan Wen. 2020. A survey on the security of blockchain systems. *Future Generation Computer Systems* 107 (2020), 841–853.
- [52] Jian-Wei Liao, Tsung-Ta Tsai, Chia-Kang He, and Chin-Wei Tien. 2019. Soliaudit: Smart contract vulnerability assessment based on machine learning and fuzz testing. In *Proceedings of the 2019 6th International Conference on Internet of Things: Systems, Management and Security*. IEEE, 458–465.
- [53] Chao Liu, Han Liu, Zhao Cao, Zhong Chen, Bangdao Chen, and Bill Roscoe. 2018. Reguard: Finding reentrancy bugs in smart contracts. In *Proceedings of the 2018 IEEE/ACM 40th International Conference on Software Engineering: Companion*. IEEE, 65–68.
- [54] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 254–269.

- [55] Yunlong Lyu, Yi Fang, Yiwei Zhang, Qibin Sun, Siqi Ma, Elisa Bertino, Kangjie Lu, and Juanru Li. 2022. Goshawk: Hunting memory corruptions via structure-aware and object-centric memory operation synopsis. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society, 1566–1566.
- [56] MakerDao. 2020. *The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System*. Retrieved June 20, 2022 from <https://makerdao.com/en/>
- [57] Lodovica Marchesi, Michele Marchesi, Giuseppe Destefanis, Giulio Barabino, and Danilo Tigano. 2020. Design patterns for gas optimization in ethereum. In *Proceedings of the 2020 IEEE International Workshop on Blockchain Oriented Software Engineering*. IEEE, 9–15.
- [58] Nicholas D. Matsakis and Felix S. Klock. 2014. The rust language. *ACM SIGAda Ada Letters* 34, 3 (2014), 103–104.
- [59] Muhammad Izhar Mehar, Charles Louis Shier, Alana Giambattista, Elgar Gong, Gabrielle Fletcher, Ryan Sanayhie, Henry M. Kim, and Marek Laskowski. 2019. Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack. *Journal of Cases on Information Technology* 21, 1 (2019), 19–32.
- [60] Alexander Mense and Markus Flatscher. 2018. Security vulnerabilities in ethereum smart contracts. In *Proceedings of the 20th International Conference on Information Integration and Web-based Applications and Services*. 375–380.
- [61] Barton P. Miller, Louis Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Communication of the ACM* 33, 12 (1990), 32–44.
- [62] Ritesh Modi. 2018. *Solidity Programming Essentials: A Beginner's Guide to Build Smart Contracts for Ethereum and Blockchain*. Packt Publishing Ltd.
- [63] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. 2019. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 1186–1189.
- [64] Bernhard Mueller. 2018. Smashing ethereum smart contracts for fun and real profit. In *Proceedings of the 9th Annual HITB Security Conference*. 54.
- [65] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).
- [66] Giang-Truong Nguyen and Kyungbaek Kim. 2018. A survey about consensus algorithms used in blockchain. *Journal of Information Processing Systems* 14, 1 (2018), 101–128.
- [67] Ivica Nikolić, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. 2018. Finding the greedy, prodigal, and suicidal contracts at scale. In *Proceedings of the 34th Annual Computer Security Applications Conference*. 653–663.
- [68] Shirin Nilizadeh, Yannic Noller, and Corina S Pasareanu. 2019. DiffFuzz: Differential fuzzing for side-channel analysis. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering*. IEEE, 176–187.
- [69] OpenZeppelin. 2017. *OpenZeppelin - SafeMath Library*. Retrieved June 20, 2022 from <https://github.com/OpenZeppelin/openzeppelin-solidity/blob/master/contracts/math/SafeMath.sol/>
- [70] Santiago Palladino. 2017. *Parity Wallet Attack Explanation*. Retrieved June 20, 2022 from <https://blog.openzeppelin.com/on-the-parity-wallet-multisig-hack-405a8c12e8f7/>
- [71] Anton Permenev, Dimitar Dimitrov, Petar Tsankov, Dana Drachsler-Cohen, and Martin Vechev. 2020. Verx: Safety verification of smart contracts. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy*. IEEE, 1661–1677.
- [72] Purathani Praitheshan, Lei Pan, Jiangshan Yu, Joseph Liu, and Robin Doss. 2019. Security analysis methods on ethereum smart contract vulnerabilities: A survey. arXiv:1908.08605. Retrieved from <https://arxiv.org/abs/1908.08605>
- [73] Quorum. 2018. *ConsenSys Quorum - ConsenSys Diligence*. Retrieved June 20, 2022 from <https://consensys.net/quorum/>
- [74] Remix. 2016. *Remix - Solidity IDE*. Retrieved June 20, 2022 from <https://ethereum.github.io/browser-solidity/>
- [75] Noama Fatima Samreen and Manar H. Alalfi. 2020. Reentrancy vulnerability identification in ethereum smart contracts. In *Proceedings of the 2020 IEEE International Workshop on Blockchain Oriented Software Engineering*. IEEE, 22–29.
- [76] Noama Fatima Samreen and Manar H. Alalfi. 2021. Smartscan: An approach to detect denial of service vulnerability in ethereum smart contracts. In *Proceedings of the 2021 IEEE/ACM 4th International Workshop on Emerging Trends in Software Engineering for Blockchain*. IEEE, 17–26.
- [77] Scilla. 2018. *MythX - ConsenSys Diligence*. Retrieved June 20, 2022 from <https://mythx.io/>
- [78] Scilla. 2018. *Scilla: Safe-By-Design Smart Contract Language*. Retrieved June 20, 2022 from <https://scilla.readthedocs.io/en/latest/>
- [79] Scilla. 2018. *Slither, the Solidity Source Analyzer*. Retrieved June 20, 2022 from <https://mythx.io/>
- [80] Kosta Serebryany. 2016. Continuous fuzzing with libfuzzer and addresssanitizer. In *Proceedings of the 2016 IEEE Cybersecurity Development*. IEEE, 157–157.
- [81] Sunbeom So, Seongjoon Hong, and Hakjoo Oh. 2021. {SmarTest}: Effectively hunting vulnerable transaction sequences in smart contracts through language {Model-Guided} symbolic execution. In *Proceedings of the 30th USENIX Security Symposium*. 1361–1378.
- [82] Solhint. 2018. *Solhint - a Open Source Project for Linting Solidity Code*. Retrieved June 20, 2022 from <https://protofire.github.io/solhint/>
- [83] Nick Szabo. 1997. Formalizing and securing relationships on public networks. *First Monday* (1997).
- [84] Xiangyan Tang, Ke Zhou, Jieren Cheng, Hui Li, and Yuming Yuan. 2021. The vulnerabilities in smart contracts: A survey. In *Proceedings of the International Conference on Artificial Intelligence and Security*. Springer, 177–190.

- [85] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. 2018. Smartcheck: Static analysis of ethereum smart contracts. In *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*. 9–16.
- [86] Palina Tolmach, Yi Li, Shang-Wei Lin, Yang Liu, and Zengxiang Li. 2021. A survey of smart contract formal specification and verification. *ACM Computing Surveys* 54, 7 (2021), 1–38.
- [87] Christof Ferreira Torres, Julian Schütte, and Radu State. 2018. Osiris: Hunting for integer bugs in ethereum smart contracts. In *Proceedings of the 34th Annual Computer Security Applications Conference*. 664–676.
- [88] Philip Treleaven, Richard Gendal Brown, and Danny Yang. 2017. Blockchain technology in finance. *Computer* 50, 9 (2017), 14–17.
- [89] Tron. 2019. *TRON - DECENTRALIZE THE WEB*. Retrieved June 20, 2022 from <https://www.tron.network/>
- [90] Petar Tsankov, Andrei Dan, Dana Drachler-Cohen, Arthur Gervais, Florian Buenzli, and Martin Vechev. 2018. Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 67–82.
- [91] Shangping Wang, Dongyi Li, Yaling Zhang, and Juanjuan Chen. 2019. Smart contract-based product traceability system in the supply chain scenario. *IEEE Access* 7 (2019), 115122–115133.
- [92] Wei Wang, Jingjing Song, Guangquan Xu, Yidong Li, Hao Wang, and Chunhua Su. 2020. Contractward: Automated vulnerability detection models for ethereum smart contracts. *IEEE Transactions on Network Science and Engineering* 8, 2 (2020), 1133–1144.
- [93] Maximilian Wohrer and Uwe Zdun. 2018. Smart contracts: Security patterns in the ethereum ecosystem and solidity. In *Proceedings of the 2018 International Workshop on Blockchain Oriented Software Engineering*. IEEE, 2–8.
- [94] Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper* 151, 2014 (2014), 1–32.
- [95] Valentin Wüstholtz and Maria Christakis. 2020. Harvey: A greybox fuzzer for smart contracts. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1398–1409.
- [96] XACML. 2019. *XACML - eXtensible Access Control Markup Language*. Retrieved June 20, 2022 from <https://tools.ietf.org/html/rfc7061>
- [97] Brent Xu, Dhruv Luthra, Zak Cole, and Nate Blakely. 2018. Eos: An architectural, performance, and economic analysis. Retrieved June 11 (2018), 2019.
- [98] Yingjie Xu, Gengran Hu, Lin You, and Chengtang Cao. 2021. A novel machine learning-based analysis model for smart contract vulnerability. *Security and Communication Networks* 2021 (2021).
- [99] Zheng Yang and Hang Lei. 2020. Lolisa: Formal syntax and semantics for a subset of the Solidity programming language in mathematical tool Coq. *Mathematical Problems in Engineering* 2020 (2020).
- [100] Jiaming Ye, Mingliang Ma, Tianyong Peng, Yun Peng, and Yinxing Xue. 2019. Towards automated generation of bug benchmark for smart contracts. In *Proceedings of the 2019 IEEE International Conference on Software Testing, Verification and Validation Workshops*. IEEE, 184–187.
- [101] Michael Yuan. 2017. *Parity Wallet Frozen Attack Explanation*. Retrieved June 20, 2022 from <https://medium.com/cybermiles/i-accidentally-killed-it-and-evaporated-300-million-6b975dc1f76b>
- [102] Z3. 2017. *The Z3 Theorem Prover*. Retrieved June 20, 2022 from <https://github.com/Z3Prover/z3/>
- [103] Yuan Zhuang, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. 2020. Smart contract vulnerability detection using graph neural network. In *Proceedings of the IJCAI*. 3283–3290.

Received 8 September 2022; revised 30 June 2023; accepted 27 January 2024